JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA [

BTech Semester-3 OOPS (Java) Project Report

Title of the project:
Stock Exchange Simulator

Submitted by:
Rohan Lal- 2401030108
Neal Pandita- 2401030110
Swastik Sangwan- 2401030098

Problem Statement

Real stock exchanges are too complex for direct experimentation.
Learners struggle to understand the basics of order matching, concurrent trading, and system constraints.
The project aims to provide a mini simulation of an exchange to demonstrate these ideas.

Introduction

Financial markets are driven by simultaneous buying and selling of assets. To understand how orders are placed, validated, and matched, students need a simplified environment that demonstrates the working of an exchange.
This project builds a **stock exchange simulator** in Java, showcasing OOP principles along with exception handling, multithreading, and file handling.

Project Objectives

The objectives of this project are:

1. To design and implement a **Java-based simulation of a stock exchange** using OOP principles.
2. To demonstrate **inheritance, polymorphism, and interfaces** through orders, accounts, and trading strategies.
3. To implement **exception handling** for invalid orders, insufficient funds, and price breaches.
4. To apply **multithreading and inter-thread communication** for simulating market data, strategy execution, and trade matching.
5. To use **file handling** for input tick data and output of trades, logs, and performance reports.
6. To provide a **learning tool** that connects programming concepts with financial market operati

Methodology

- **Core Modules:** OrderBook, Orders (Market & Limit), Exchange, Accounts, Trading Strategies.
- **OOP Concepts:** Inheritance, polymorphism, interfaces for flexible strategies.
- **Exception Handling:** For invalid trades, insufficient funds, and out-of- range prices.
- **Multithreading:** Threads for market feed, strategies, order intake, and trade logging.
- **Producer–Consumer Model:** Queues to pass data between threads.
- **File Handling:** Input tick data, output trade and PnL reports.

Expected Outcomes

- A working web app **exchange simulator**.
- Demonstration of OOP, exception handling, multithreading, and file I/O in practice.
- Educational value for both programming and finance concepts.

OOP Concepts Used

2.1 Classes

- `Account`
- `Trade`
- `StockExchange`
- `WebServer`
- `BuyHandler`
- `SellHandler`
- `PortfolioHandler`
- `Util`
- Frontend UI scripts (`script.js`, `charts.js`)

2.2 Objects

Accounts, trades, exchange instances, and HTTP handler objects.

2.3 Encapsulation

Fields like `cashBalance`, `holdings`, and `tradeHistory` are private and accessed through methods.

2.4 Abstraction

The `StockExchange` class abstracts:

- Buying logic
- Selling logic
- Price lookup
- Portfolio formatting

2.5 Composition

- `Account` contains multiple `Trade` objects.
- `StockExchange` contains multiple accounts.

2.6 Polymorphism

Not required heavily, but HTTP handlers all implement `HttpHandler`.

CLASS DESCRIPTIONS (Backend)

`Trade` Class
Purpose:
Represents a single stock trade (buy/sell).
Attributes:

- stockSymbol
- quantity
- price
- `type` (BUY/SELL)
- timestamp

Key Methods:

- `toString()` – nicely formats trade details.

Account Class
Purpose:
Holds user portfolio, cash, and trade history.
Attributes:

- id
- name
- cashBalance
- `holdings` (HashMap<String, Integer>)
- tradeHistory (List<Trade>)

Key Methods:

- buy(symbol, qty, price)
- sell(symbol, qty, price)

- `portfolio()` – displays portfolio and trade history.

Exchange Class
Purpose:
Central class that manages accounts, stock prices, and trade operations.
Attributes:
- `prices` (stock symbol → price)
- `accounts` (account ID → Account)

Key Methods:
- getAccount(id)
- getPrice(symbol)
- buy(id, symbol, qty)
- sell(id, symbol, qty)
- portfolio(id)

WebServer Class
Purpose:
Creates and starts the backend HTTP server.
Responsibilities:
- Registers routes:
    - /buy
    - /sell
    - /portfolio
- Runs the server on port 8080.

Handler Classes

BuyHandler

Reads GET parameters → calls exchange.buy() → returns result.

SellHandler

Reads GET parameters → calls exchange.sell() → returns result.

PortfolioHandler

Calls exchange.portfolio(accountId).

WORKFLOW

1️⃣ Frontend → Sends Request

User clicks BUY / SELL / PORTFOLIO button (or JS triggers fetch).

Frontend sends a GET request to server with query parameters:

/buy?acc=0&symbol=AAPL&qty=5

/sell?acc=1&symbol=TSLA&qty=3

/portfolio?acc=0

2️⃣ WebServer.java → Receives Request

WebServer is listening on port 8080.

It checks which endpoint was called:

/buy → BuyHandler

/sell → SellHandler

/portfolio → PortfolioHandler

3️⃣ Handler Takes Over (Example: BuyHandler)

Handler receives HttpExchange object.

It extracts the query string from the URL.

It uses Util.parseQuery() to convert:

acc=0&symbol=AAPL&qty=5

into:

{"acc":"0", "symbol":"AAPL", "qty":"5"}

---

4 Handler → Calls StockExchange Methods

BuyHandler calls:

StockExchange.buy(id, symbol, qty)

SellHandler calls:

StockExchange.sell(id, symbol, qty)

PortfolioHandler calls:

StockExchange.portfolio(id)

---

5 StockExchange → Main Business Logic

Checks if account exists.

Looks up current stock price.

For buy:

Verifies account has enough cash.

Deducts money.

Adds shares.

Creates a Trade record.

For sell:

Checks if account has enough shares.

Removes shares.

Adds cash.

Creates Trade record.

For portfolio:

Builds a readable formatted portfolio string.

---

6 Account.java → Updates Internal Data

Updates cashBalance

Updates holdings HashMap

Adds Trade object to tradeHistory

Returns a complete portfolio string

---

7 Trade.java → Creates Trade Record

Stores:

STOCK symbol

Quantity

Price

Type (BUY/SELL)

Timestamp

Used for portfolio history display.

---

8 Result Returned from StockExchange → Handler

Handler receives the final result string.

Example:

Buy Successful

Account: Rohan (ID 0)
Cash Balance: 40500.0

Holdings:
  AAPL: 5

Trade History:
  BUY | AAPL | Qty: 5 | Price: 190.50 | Time: ...

9️⃣ Handler Sends HTTP Response

Converts result to UTF-8 bytes.

Adds:

Access-Control-Allow-Origin: *

(So frontend can access the backend.)

Sends HTTP response with status 200.

Writes response body.

Closes the stream.

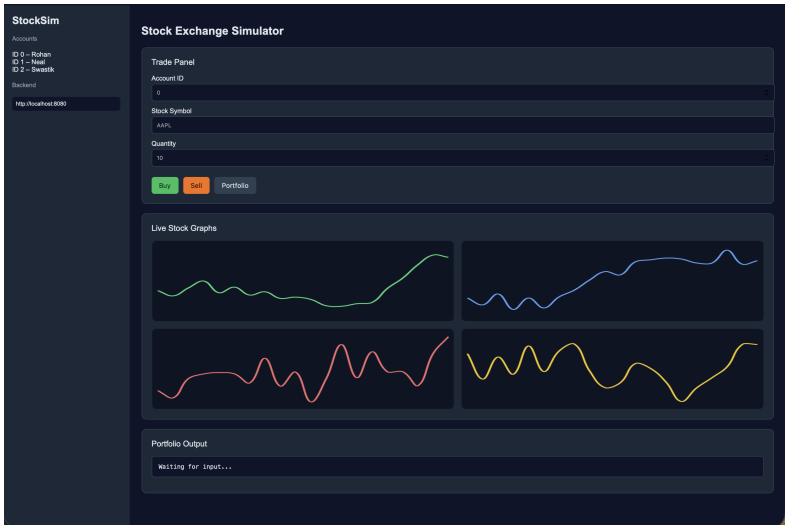
🔟 Frontend Receives Response

Frontend JS receives .text() response.

Displays it inside a UI element (e.g., <pre>,).

User sees updated portfolio or error message

SNAPSHOTS


StockSim
Accounts
ID 0 – Rohan
ID 1 – Neal
ID 2 – Swastik
Backend
http://localhost:8080
Stock Exchange Simulator
Trade Panel
Account ID
0
Stock Symbol
AAPL
Quantity
10
Buy
Sell
Portfolio
Live Stock Graphs
Portfolio Output
Waiting for input...

StockSim
Accounts
ID 0 – Rohan
ID 1 – Neal
ID 2 – Swastik
Backend
http://localhost:8080
Stock Exchange Simulator
Trade Panel
Account ID
2
Stock Symbol
MSFT
Quantity
3
Buy
Sell
Portfolio
Live Stock Graphs
Portfolio Output
Sell Successful
Account: Swastik (ID 2)
Cash Balance: 32787.4
Holdings:
MSFT: 96
GOOG: 12
Trade History:
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:49
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:55
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:58

StockSim
Accounts
ID 0 – Rohan
ID 1 – Neal
ID 2 – Swastik
Backend
http://localhost:8080
Stock Exchange Simulator
Trade Panel
Account ID
2
Stock Symbol
MSFT
Quantity
5
Buy
Sell
Portfolio
Live Stock Graphs
Portfolio Output
Buy Successful
Account: Swastik (ID 2)
Cash Balance: 31737.1
Holdings:
MSFT: 99
GOOG: 12
Trade History:
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:49
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:55
BUY | GOOG | Qty: 4 | Price: 2800.25 | Time: 2025-11-14 00:11:58

FUTURE IMPROVEMENTS

1 Persistent Storage

- Save accounts, trades, and holdings in:
    - JSON
    - Text file
    - MySQL or SQLite database

2 User Authentication

- Login system
- Session handling
- JWT tokens

3 Real Market API Integration

Replace demo prices with:

- Yahoo Finance API
- AlphaVantage
- Finnhub

This would allow real stock price updates.

4 Real-Time WebSockets

- Update charts dynamically from backend
- Show live ticker movements

5 Better UI Components

- Candlestick charts
- Price tickers
- Animations for buy/sell confirmation

6 Add More Trading Features

- Limit orders
- Stop-loss
- P&L calculation
- Chart history
- Risk analysis panel

7 Mobile-Responsive UI

Make the UI adapt perfectly for:

- iPhones
- Tablets
- Small laptops

CONCLUSION

This project successfully demonstrates:

- Use of OOP principles
- A fully functional **Java backend server**
- A clean, modern HTML/CSS/JS dashboard
- Live graph visualization
- Integration of backend and frontend through HTTP APIs

It fulfills all requirements for a college-level Object-Oriented Programming project and provides room for future expansion toward a professional trading platform.